

테스트 - 타임 메모리 계열 — Titans 에서 Sleep 까지 , 그리고 서빙 비용

학습을 처음 보는 inference·systems 엔지니어를 위한 6 편 (Titans·Miras·Atlas·TNT·Nested Learning·Sleep) 해부
+ 배포 비용 모델링

이승호 (SAIT) · 2 시간 발표 + 30 분 Q&A

오늘 다룰 것

A. Background (학습 입문)

A1. 학습 한 바퀴 — 2-layer MLP:

forward→loss→backward→optimizer→update

A2. Optimizer 의 역사 (수식과 함께)

계보 지도 (제공 이미지) →

GD · SGD · Momentum

Adagrad · RMSProp · Adam · AdamW

A3. 대안 계열

linear attention

DeltaNet → Gated DeltaNet

SSM · Mamba-2

B. 논문 6 편 (발명→구현→증명)

B1. Titans — deep neural memory

surprise · momentum · forget, MAC/MAG/MAL

B2. Miras — 4 축 설계공간

메모리 = online 최적화

B3. Atlas — Omega rule · 용량 · Muon

B4. TNT — chunk · 2-stage · global/local · Q-K

B5. Nested Learning / HOPE

optimizer=memory · self-mod · CMS

B6. Sleep — offline consolidation

Knowledge Seeding · Dreaming

C. System modeling (우리 기여)

C1. pair thesis — decode=memory-bound

C2. 8 개 가속기 크로스 측정

C3. HOPE-block roofline

+ zHBM · HBM-PIM · SRAM scaling

C4. scaling 청사진

부록 . QA 모음 (교차 · 논문별)

이 세미나의 목적 · 범위 · 주의

목적 · 방향

inference·systems 엔지니어가 test-time memory 계열 (Titans~Sleep) 을 shape· 비용 수준으로 이해한다 . 각 논문의 핵심 발명을 먼저 보고
→ 어떻게 구현 · 증명되는지 → 논문 간 framing 이 이어지는 흐름 → 마지막에 이 배포 형태가 서빙에서 얼마나 드는지 (우리 기여).

미리 알아둘 3 가지 (읽는 규칙)

① 학습 (pre-training) 은 다루지 않는다

Titans 기반 모델을 처음부터 훈련하는 과정은 범위 밖 . 이 계열의 test-time 동작과 서빙에 집중한다 . (Background 의 학습 설명은 ' 무엇을 배우는가 ' 를 읽기 위한 최소한 .)

② 논문에 구체 수치가 빠진 곳이 많다

모델 파라미터 shape· 차원 · 학습 하이퍼 등 구체값이 명시 안 된 부분이 흔하다 . 그림 · 설명에서 추론한 것은 ' 추론 ' 으로 , 본문에 있는 것은 ' 명시 ' 로 구분해 표시한다 .

③ 논문 1 편만으로 불완전하다

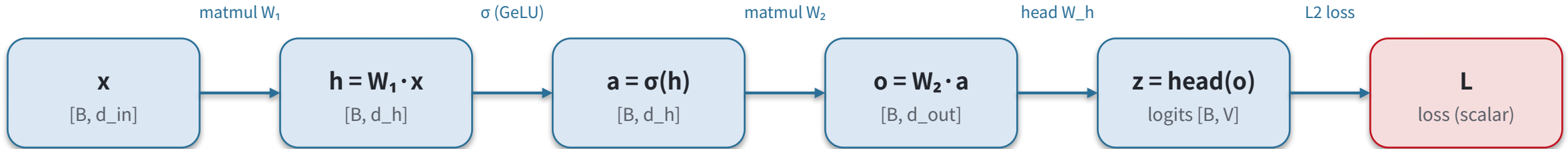
한 논문의 설명이 스스로 완결되지 않아 , 다른 논문 (특히 HOPE) 을 같이 봐야 이해되는 부분이 많다 . 예 : Titans 의 투영 분리는 HOPE 에서야 명시된다 .
그래서 6 편을 하나의 흐름으로 읽는다 .

PART A

Background — 학습이 처음이라면

2-layer MLP + head: forward → loss → backward → optimizer → update

① forward



batch B 개 샘플 동시 처리 · loss 는 B 개 평균 (또는 합) 으로 두적

② loss.backward() — autograd 역전파

$\partial L / \partial z \rightarrow \partial L / \partial W_2, \partial L / \partial a \rightarrow \partial L / \partial h \rightarrow \partial L / \partial W_1, \partial L / \partial \text{head}$. 각 파라미터 W 의 gradient $g = \partial L / \partial W$ 를 구해 W.grad 에 채운다 (순전파의 역순, 체인룰).

③ optimizer.step() — AdamW (gradient g 를 받아 '얼마나 · 어느 방향' 결정)

$m \leftarrow \beta_1 \cdot m + (1 - \beta_1) \cdot g$ (1 차 모멘텀 : 방향의 관성)
 $v \leftarrow \beta_2 \cdot v + (1 - \beta_2) \cdot g^2$ (2 차 모멘텀 : 좌표별 스케일)
 $\hat{m}, \hat{v} = \text{편향보정}$; $\Delta W = -\eta \cdot \hat{m} / (\sqrt{\hat{v}} + \epsilon)$ - $\eta \cdot \lambda \cdot W \leftarrow \text{decoupled decay}$

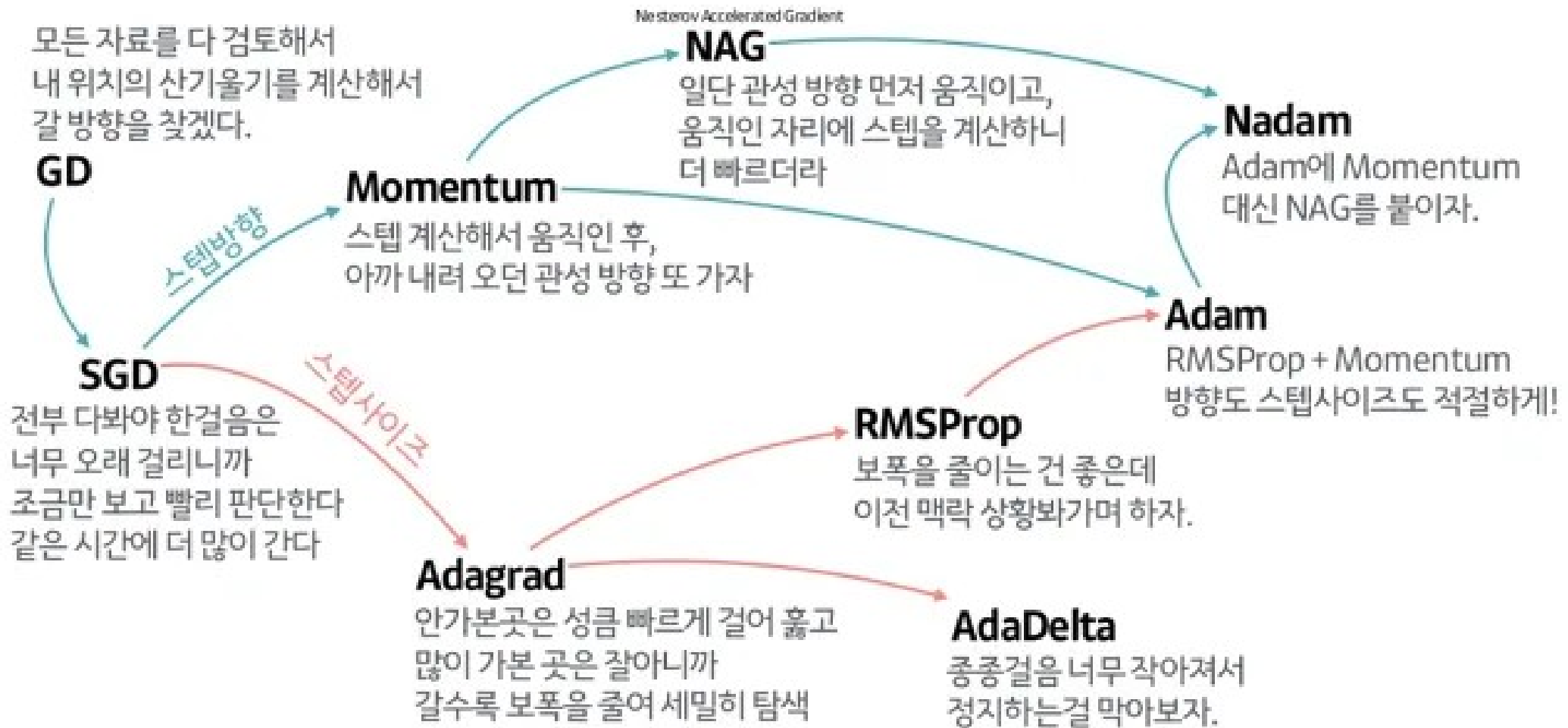
④ $W \leftarrow W + \Delta W$

W_1, W_2, head 를 갱신.
 optimizer.zero_grad() 로 g 비우고
 다음 batch 로 → 반복.

핵심 한 문장 .

학습 = ①forward 로 예측 → ② backward 로 'gradient(어디가 틀렸나)' → ③optimizer 로 'gradient 를 기억 · 가공 (m,v) 해 갱신량' → ④weight 이동 . 이 계열은 ②③④를 추론 중에도 돌린다 .

한 장으로 보는 직관 지도



파란 줄기 = '어느 방향으로 '(관성) 를 고치는 흐름 · 초록 줄기 = ' 얼마나 (보폭)' 를 고치는 흐름 → Adam 이 둘을 합침 . 다음 장부터 이 중
GD·SGD·Momentum·Adagrad·RMSProp·Adam·AdamW 를 1 페이지씩 수식과 함께 .

GD · Gradient Descent (batch)

뿌리 — 가장 정확한 한 걸음

직관 (왜 이렇게 ?)

" 모든 자료를 다 검토해서 , 내 위치의 산 기울기를 계산해 갈 방향을 찾았다 ."

갱신 규칙 (수식)

$$g_t = \frac{1}{N} \sum_{i=1}^N \nabla \ell_i(\theta_t) \quad \text{전체 } N \text{ 개 데이터의 평균 gradient}$$

$$\theta_{t+1} = \theta_t - \eta g_t \quad \eta = \text{learning rate (보폭)}$$

이 optimizer 가 해결한 것

손실을 가장 빨리 줄이는 방향 (가장 가파른 내리막) 을 데이터 전체로 정확히 계산한다 .

남은 문제 → 다음

한 스텝마다 데이터 N 개를 전부 봐야 함 → 너무 느리다 . → SGD

SGD · Stochastic Gradient Descent

파란 줄기 — 속도

직관 (왜 이렇게 ?)

" 전부 다 봐야 한 걸음은 너무 오래 걸리니까 , 조금만 보고 빨리 판단한다 . 같은 시간에 더 많이 간다 ."

갱신 규칙 (수식)

$$g_t = \nabla \ell_B(\theta_t) \quad \text{미니배치 } B \text{ 개만 사용 } (|B| \ll N)$$

$$\theta_{t+1} = \theta_t - \eta g_t$$

이 optimizer 가 해결한 것

스텝당 비용이 급감 → 같은 시간에 훨씬 많은 스텝 . gradient noise 가 얇은 지역최소 탈출에도 도움 .

남은 문제 → 다음

gradient 가 매 스텝 출렁여 지그재그 . 방향도 보폭도 여전히 그대로 .
→ Momentum · Adagrad

Momentum · SGD with Momentum

파란 줄기 — '스텝 방향' 개선

직관 (왜 이렇게 ?)

" 스텝을 계산해서 움직인 후 , 아까 내려오던 관성 방향으로 또 가자 ."

갱신 규칙 (수식)

$$v_t = \mu v_{t-1} + g_t \quad \mu \approx 0.9, \text{ 과거 방향의 관성 (누적)}$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

이 optimizer 가 해결한 것

일관된 방향은 누적 가속하고 출렁이는 성분은 상쇄 → 지그재그 완화 , 좁은 골짜기를 빠르게 통과 .

남은 문제 → 다음

보폭 (스케일) 은 아직 모든 좌표가 동일 . ' 좌표별 보폭 ' 은 아래 초록 줄기 (Adagrad→RMSProp) 가 맡고 , 뒤에서 Adam 이 방향과 합친다 .

Adagrad · Adaptive Gradient

초록 줄기 — '스텝 사이즈' 개선

직관 (왜 이렇게 ?)

" 안 가 본 곳은 성큼 빠르게 훑고 , 많이 가 본 곳은 잘 가니까 갈수록 보폭을 줄여 세밀히 탐색 ."

갱신 규칙 (수식)

$$G_t = G_{t-1} + g_t^2 \quad \text{좌표별 gradient}^2 \text{ 누적}$$

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{G_t} + \epsilon} \quad \text{좌표마다 다른 보폭}$$

이 optimizer 가 해결한 것

좌표마다 다른 learning rate — 드물게 큰 gradient(희소 feature) 는 크게 , 자주 큰 좌표는 작게 .

남은 문제 → 다음

G_t 가 단조 증가 → 분모가 계속 커져 보폭이 0 으로 죽는다 (학습 정지). → RMSProp

RMSProp · Root Mean Square Propagation

초록 줄기 — EMA 로 보완

직관 (왜 이렇게 ?)

" 보폭을 줄이는 건 좋은데 , (전부 누적하지 말고) 최근 맥락을 봐 가며 하자 ."

갱신 규칙 (수식)

$$\mathbb{E}[g^2]_t = \rho \mathbb{E}[g^2]_{t-1} + (1 - \rho) g_t^2 \quad \rho \approx 0.9, \text{ 최근값 지수이동평균 (EMA)}$$

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{\mathbb{E}[g^2]_t + \epsilon}} \quad \text{분모가 무한정 커지지 않음}$$

이 optimizer 가 해결한 것

합 대신 EMA → 분모가 무한정 커지지 않아 보폭이 죽지 않는다 .
비정상 (non-stationary) 목표에도 안정 .

남은 문제 → 다음

방향 (momentum) 은 아직 안 씀 . 초기 (0 에서 시작) 편향 보정도 없다 . → Adam

Adam · Adaptive Moment Estimation

두 줄기의 합류 ★

직관 (왜 이렇게 ?)

"RMSProp(보폭) + Momentum(방향) 을 합치자 — 방향도 스텝 사이즈도 적절하게 !"

갱신 규칙 (수식)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad \text{1 차 모멘텀 : 방향}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad \text{2 차 모멘텀 : 좌표별 보폭}$$

$$\hat{m} = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v} = \frac{v_t}{1 - \beta_2^t} \quad \text{초기 0- 편향 보정}$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$$

이 optimizer 가 해결한 것

방향 (m) + 좌표별 보폭 (v) + 편향 보정을 한 번에 . 튜닝 거의 없이 잘 동작 → 사실상 표준 .

남은 문제 → 다음

L2 weight decay 를 gradient 에 더하면 $1/\sqrt{\hat{v}}$ 로 나뉘어 좌표별로 왜곡된다 . → AdamW

AdamW ★ · Adam with decoupled Weight decay

종착점 — 오늘의 표준

직관 (왜 이렇게 ?)

"weight decay(정규화) 를 gradient 에 섞지 말고 , weight 에서 ' 직접 ' 빼자 — 그래야 좌표마다 똑같이 적용된다 ."

갱신 규칙 (수식)

$m_t, v_t, \hat{m}, \hat{v}$ (as in Adam) decay 를 gradient 에 넣지 않음

$$\theta_{t+1} = \theta_t - \eta \left(\frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon} + \lambda \theta_t \right) \quad \lambda \theta \text{ 항을 분리 (decoupled)}$$

이 optimizer 가 해결한 것

decay 가 adaptive 스케일 ($1/\sqrt{\hat{v}}$) 에 섞이지 않아 정규화가 일관 → 일반화 ↑ . 오늘날 LLM 학습의 표준 .

남은 문제 → 다음

여기까지가 ' 표준 optimizer '. 이 계열 논문은 이 optimizer 자체를 memory 로 재해석한다 (HOPE). → Part B

왜 softmax attention 을 벗어나나 — sequence 를 ' 상태 ' 로 압축하는 계열

softmax attention

- 문맥이 길수록 KV 캐시가 선형으로 증가 (메모리 폭발)
- 토큰마다 전체를 다시 봄 → 계산 $O(n^2)$
- 무엇을 ' 기억 ' 할지 압축이 없다 — 전부 보관



고정 크기 ' 상태 ' S 로 압축하는 RNN 류

토큰마다 상태 S 를 ' 어떤 규칙으로 ' 갱신하느냐가 계열을 가른다 . 이 관점이 곧 Titans 의 **memory update(=test-time 학습)** 로 이어진다 .

Linear Attention

S 를 더하기만 (additive)
누적 간섭이 문제

DeltaNet → Gated

delta rule 로 덮어쓰기 · 수정
+ forget gate

SSM · Mamba-2

구조화된 선형 recurrence
선택적 상태 갱신

공통 렌즈 — 다음 3 계열을 관통하는 한 문장

모두 " 상태 S 를 토큰마다 갱신하는 선형 recurrence" 다 . 갱신 규칙의 차이일 뿐 — 더하기 (linear), 덮어쓰기 (delta), 잊기 (gated), 구조화 (SSM).
→ Titans 는 이 갱신을 아예 ' 작은 신경망 메모리를 gradient 로 학습 ' 하는 것으로 승격시킨다 (Part B).

상태 S 에 ' 쓰는 규칙 ' 을 바꿔 온 역사

Linear Attention

$\phi(k)v^T$ 를 계속 더함
= fast weights (외적 누적)

$$S_t = S_{t-1} + \phi(k_t) v_t^T, \quad o_t = \phi(q_t)^T S_t$$

DeltaNet

delta rule: 예측오차만큼
기존 기억을 ' 수정 / 덮어쓰기 '

$$S_t = S_{t-1} - \beta_t (S_{t-1} k_t - v_t) k_t^T$$

Gated DeltaNet

+ forget gate α_t (0~1)
오래된 기억을 잊어 용량 확보

$$S_t = \alpha_t S_{t-1} - \beta_t (S_{t-1} k_t - v_t) k_t^T$$

연결

" 예측오차 ($S k - v$) 만큼 상태를 고친다 " 는 delta rule 은 사실 손실 $\| S k - v \|^2$ 에 대한 gradient step 한 번과 같다 . 여기에 forget(α) · momentum 을 얹으면 → 바로 Titans 의 memory 갱신식이 된다 .

구조화된 선형 recurrence, 그리고 attention 과의 이중성

SSM (S4)

고정된 A,B,C (시간 불변)
긴 의존성을 구조화된 A 로

$$h_t = A h_{t-1} + B x_t, \quad y_t = C h_t$$

Mamba (S6)

선택적 (selective): 입력에 따라
무엇을 상태에 넣을지 결정

$$(A_t, B_t, C_t) = f(x_t) \text{ — input-dependent}$$

Mamba-2 (SSD)

state-space duality:
attention $\leftrightarrow$ SSM 은 같은 것의 두 형태

$$Y = (L \circ (CB^{\top})) X \iff \text{linear SSM recurrence}$$

요지

SSM 은 ' 상태 갱신 규칙을 구조화된 선형식 ' 으로 본다 . Mamba-2 의 duality 는 attention 과 recurrence 가 등가임을 보여 , "sequence layer = 상태를 갱신하는 하나의 규칙 " 이라는 이번 세미나의 렌즈를 뒷받침한다 .

PART B

논문 6 편 — 핵심 발명 먼저 , 그다음 어떻게 증명했나

Titans — test-time 에 학습되는 신경망 메모리

핵심 발명 / 주장

1

문맥을 KV 에 쌓지 말고 , test-time 에 ' 학습되는 ' deep neural memory(작은 MLP) 에 압축한다
attention 의 $O(n^2) \cdot KV$ 폭발 대신 , 고정 크기 신경망이 무엇을 기억할지 스스로 학습

2

무엇을 · 얼마나 기억할지 = surprise(gradient) + momentum(과거 surprise) + forgetting(decay)
예측이 놀라운 (=gradient 큰) 토큰을 더 기억 , 관성으로 이어가고 , 오래된 건 잊는다

3

메모리를 아키텍처에 꽂는 3 가지 방식 : MAC · MAG · MAL
persistent(작업기억) + core(attention) + contextual(장기 메모리) 의 결합 방식 차이

논거 흐름

linear/KV 의 용량 · 간섭 한계 → 메모리를 ' 학습되는 MLP' 로 → 무엇을 배우나 $=k \rightarrow v$ 연상 ($\| M(k)-v \|^2$) → 얼마나 $=surprise+momentum+forget$ → 병렬화
 $=momentum$ 을 associative scan 으로 → 어디에 꽂나 =MAC/MAG/MAL

증명 (뒤에서 논문 그림 · 표로)

언어모델링 perplexity · NIAH(needle) · BABILong(초장문 추론) 에서 baseline 상회 , 메모리 깊을수록 개선

메모리 = 작은 MLP 를 test-time 에 gradient 로 갱신

무엇을 배우나 — key→value 연상

토큰 $x_t \rightarrow k_t = x_t W_K, v_t = x_t W_V$. 메모리 M 이 k_t 를 넣으면 v_t 가 나오도록 학습.

$$\ell(M_{t-1}; x_t) = \|M_{t-1}(k_t) - v_t\|^2$$

갱신 규칙 (η, θ, α 는 데이터 의존)

$$u_t = \nabla_M \ell(M_{t-1}; x_t) \quad \text{momentary surprise = gradient}$$

$$S_t = \eta_t S_{t-1} - \theta_t u_t \quad \text{surprise memory (momentum, } \eta = \text{관성)}$$

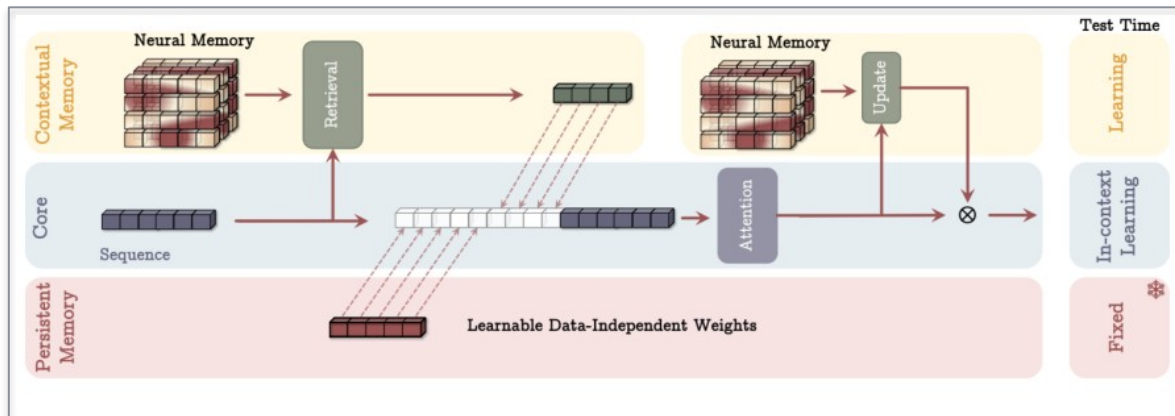
$$M_t = (1 - \alpha_t) M_{t-1} + S_t \quad \text{forgetting gate } \alpha \text{ 로 오래된 기억 감쇠}$$

$$y_t = M_t(q_t) \quad \text{읽기 : query } q_t \text{ 를 갱신된 메모리에 통과}$$

핵심 포인트

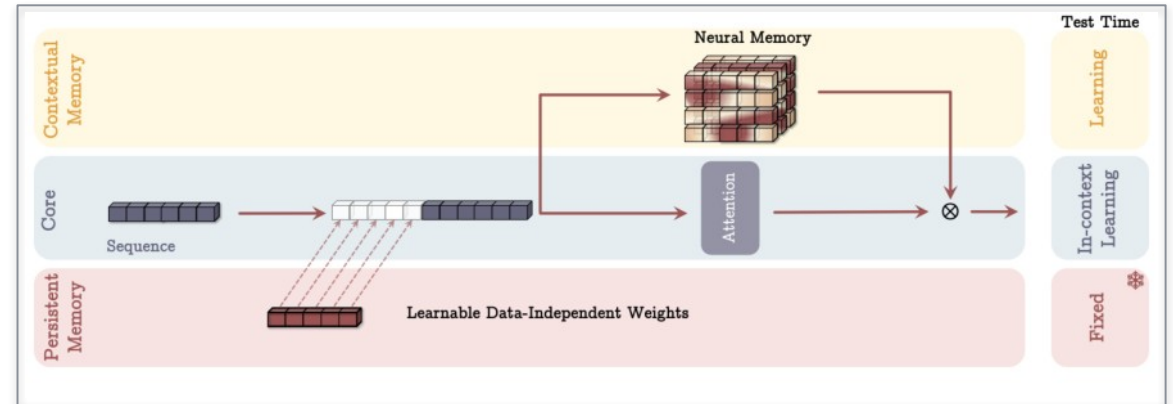
이 3 식은 A2 의 'momentum + weight decay 가 붙은 gradient descent' 와 정확히 같은 모양이다 — 다만 대상이 모델 파라미터가 아니라 '추론 중 메모리 M ' 이다. 즉 학습 (training) 의 도구를 추론 (inference) 안으로 가져왔다.

MAC · MAG · MAL — 메모리를 아키텍처에 넣는 3 가지



MAC: 메모리를 검색해 attention 의 ' 문맥 ' 으로 넣음 (persistent+core+contextual)

출처 : 2501.00663 Fig.2



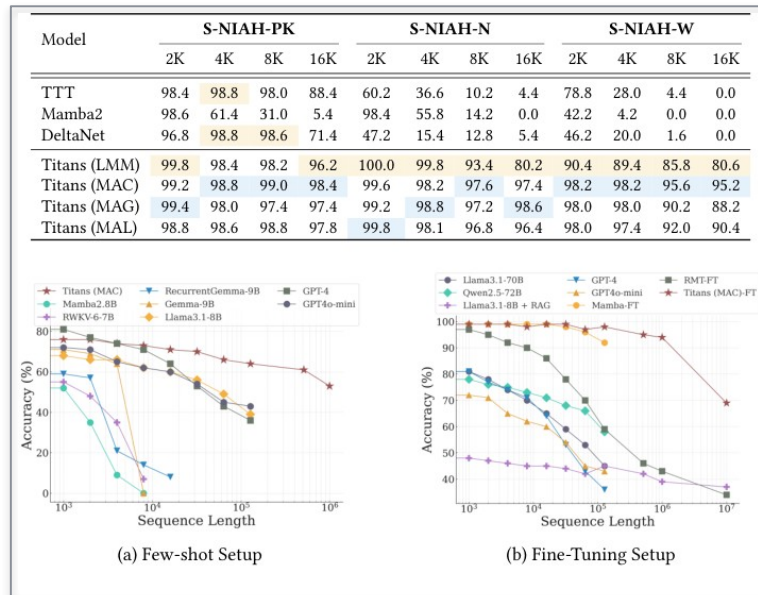
MAG: 장기 메모리 분기와 sliding-window attention 분기를 게이트로 결합

출처 : 2501.00663 Fig.4

세 방식의 차이

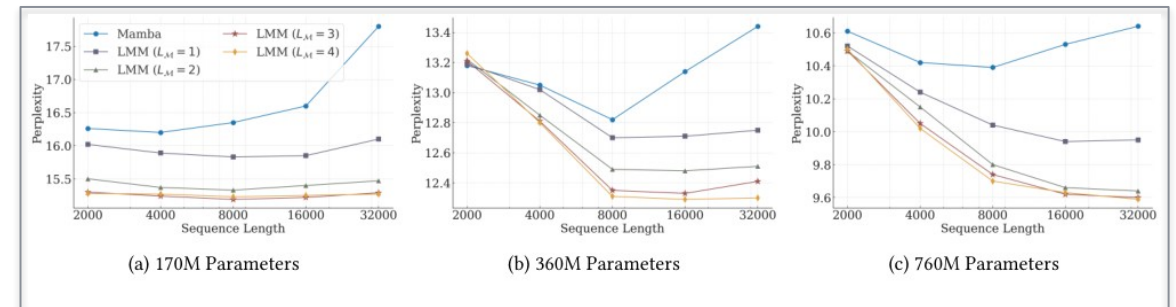
MAC= 메모리 출력을 문맥으로 concat(가장 강함 , 느림) · MAG= 게이트로 혼합 · MAL= 메모리를 attention 앞 레이어로 . 공통 : persistent memory(입력 독립 작업기억) + core(단기 attention) + long-term memory(위 식으로 학습). MAL 은 그림 생략 (레이어형).

초장문에서 baseline 상회, 메모리가 깊을수록 좋아진다



BABILong: Titans(MAC) 가 GPT-4·Mamba·RMT 등보다 긴 문맥에서 우위

출처: 2501.00663 Fig.6



메모리 깊이 (MLP 층수) ↑ → perplexity ↓, 특히 긴 문맥에서 이득

출처: 2501.00663 Fig.7

읽는 법

왼쪽: 문맥 길이가 늘어도 정확도 유지 (= 압축이 실제로 기억을 보존). 오른쪽: '메모리를 깊게 = 표현력 ↑' 이 실측으로 확인 → 뒤 논문들이 '메모리 용량 · 표현력' 을 키우는 방향으로 나아가는 근거.

Miras — 메모리는 하나의 online 최적화 ; 4 축 설계공간

핵심 발명 / 주장

1

sequence layer = 하나의 online 최적화 문제로 통일해 볼 수 있다

Titans · DeltaNet · linear attention 등은 모두 '어떤 목적을 어떤 규칙으로 최소화하나'의 특수해

2

설계는 4 개의 축으로 분해된다 : attentional bias · retention · architecture · algorithm

① 무엇을 기억 (손실) ② 무엇을 잊기 (retention) ③ 메모리 구조 ④ 갱신 알고리즘

3

이 축에서 새 인스턴스를 만든다 : Moneta · Yaad · Memora (FTRL · Learning-Retaining)

발명이라기보다 '설계 공간의 존재 증명 + 각 축의 효과를 실험으로 확인'

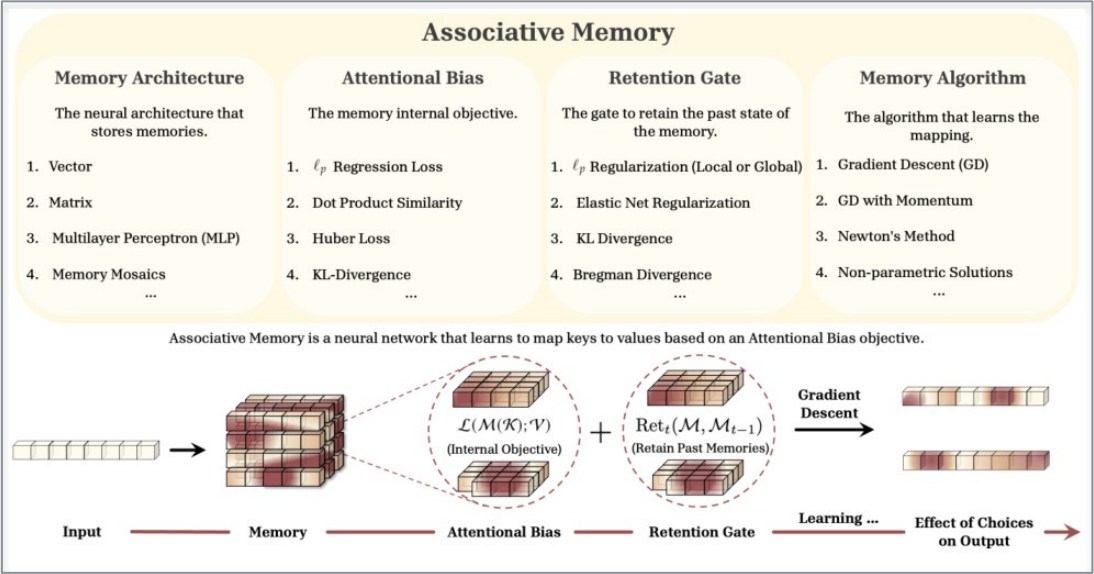
논거 흐름

Titans 재해석 (surprise=attentional bias, forget=retention) → 축을 일반화 → 손실을 L2 에서 Lp/robust 로 , retention 을 다양하게 → 새 조합
Moneta/Yaad/Memora → 축별 ablation

증명 (뒤에서 논문 그림 · 표로)

모델 크기 · 문맥 길이 스케일링 (Fig.3), 손실차수 p · retention q 의 효과 (Fig.4) 로 각 축이 실제로 성능을 가른다

메모리를 4 개의 독립적 '선택' 으로 분해



Miras 프레임워크 : 4 가지 핵심 선택으로 sequence layer 를 구성
출처 : 2504.13173 Fig.1

① Attentional bias

무엇을 기억 ? = 손실 함수 . Titans 의 $\| M(k)-v \|^2$ 는 한 예 . Lp·robust 로 바꿀 수 있다 .

② Retention gate

무엇을 잊나 ? = 정규화 / 감쇠 . forget gate 가 여기 속한다 .

③ Memory architecture

메모리의 구조 : vector · matrix · deep MLP.

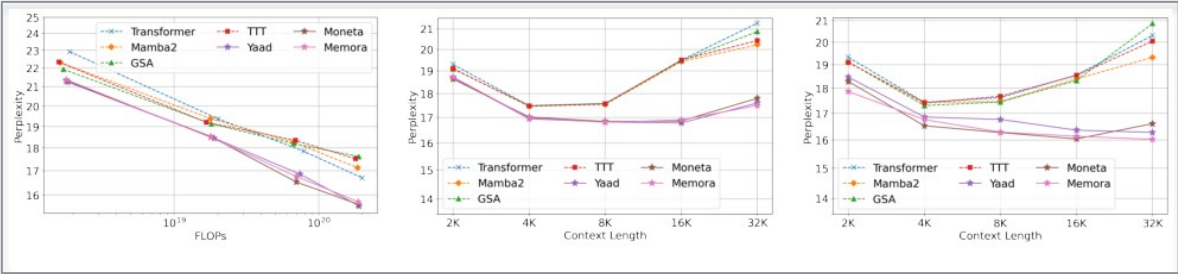
④ Learning algorithm

어떻게 갱신 ? GD · momentum · FTRL · Learning-Retaining.

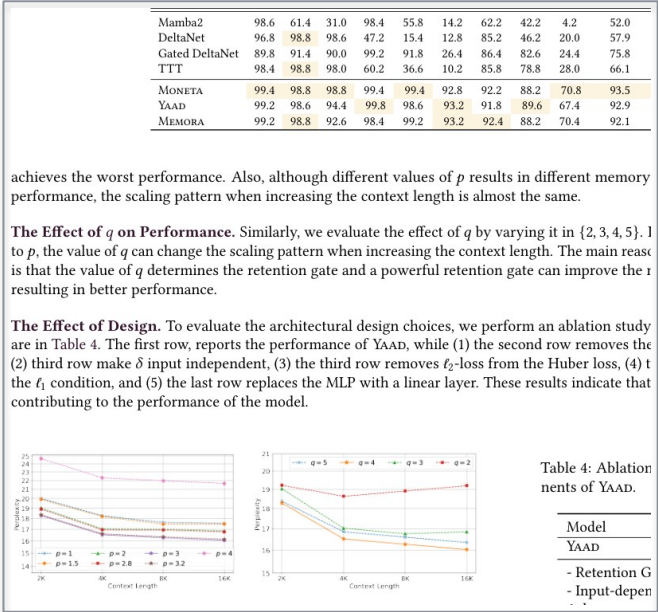
의의

Titans 를 특수점으로 품는 지도를 그려 , ' 무엇을 바꾸면 무엇이 좋아지나 ' 를 조직적으로 탐색 가능하게 했다 . 이후 Atlas·HOPE 는 이 축들을 각자 밀어붙인 결과다 .

축을 움직이면 성능이 예측대로 변한다



모델 크기 · 문맥 길이에 따른 스케일링 — Miras 변형들이 안정적으로 개선
출처 : 2504.13173 Fig.3



손실 차수 p · retention q 의 효과 — 축 선택이 문맥 길이별 성능을 가른다
출처 : 2504.13173 Fig.4

읽는 법
각 축을 독립적으로 흔들었을 때 성능이 단조 · 예측 가능하게 반응 → ' 설계 공간 ' 이 실재함을 뒷받침 . 이 관점이 다음 Atlas(용량 축) 와 HOPE(알고리즘 축) 의 출발점 .

Atlas — 토큰이 아니라 ' 문맥 ' 을 기억하고 , 용량을 키운다

핵심 발명 / 주장

1

Omega rule: 개별 토큰이 아니라 sliding-window ' 문맥 전체 ' 를 한 번에 기억

$y_{\{t,i\}}$ 가중으로 최근 c 개 토큰을 함께 최적화 — 'learning to memorize the context'

2

feature map ϕ_p 로 메모리 용량을 matrix 한계 ($\sqrt{\text{params}}$) 너머 d^p 까지 확장

polynomial feature map 으로 표현력 $\uparrow$ (softmax 는 사실상 무한 용량의 극단)

3

메모리 학습을 GD 가 아니라 Muon(근사 2 차) optimizer 로

곡률 정보를 써 더 잘 · 빠르게 메모리를 적합 — A2 의 optimizer 발전을 메모리 학습에 적용

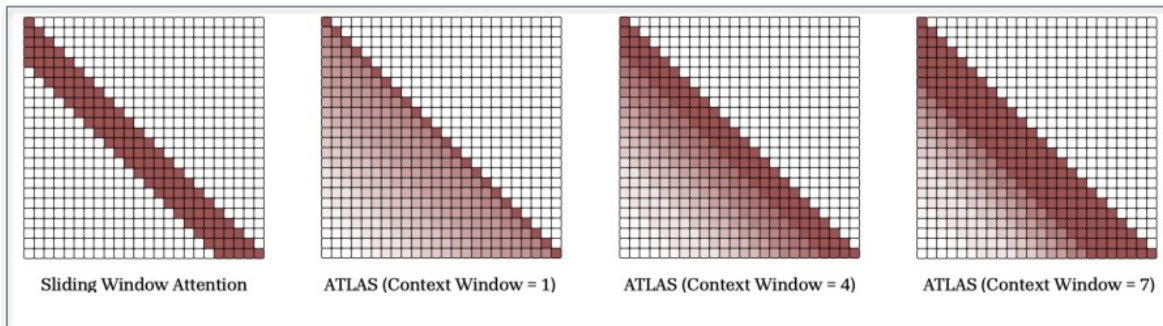
논거 흐름

메모리 용량이 병목 (Titans Fig.7 의 연장) $\rightarrow$ Hopfield 용량 이론 $\rightarrow$ feature map 투영으로 확장 $\rightarrow$ 문맥 단위 목적 (Omega) $\rightarrow$ 최적화도 Muon 으로 $\rightarrow$ DeepTransformers/Dot 변형

증명 (뒤에서 논문 그림 · 표로)

BABILong 에서 Titans 초과 (Fig.4), associative recall 용량 실험 (Fig.7), 문맥 · FLOPs 스케일링 (Fig.8)

문맥 단위 기억 (Omega) + 용량 확장 (feature map)



SWA vs Atlas/Omega: 문맥 내 토큰 의존성을 더 넓게 연결

출처: 2505.23735 Fig.2

Omega rule (문맥 단위 목적)

최근 c 개 토큰을 γ 가중으로 한 번에 적합 :

$$\min_M \sum_{i=t-c+1}^t \gamma_{t,i} \|M(k_i) - v_i\|^2$$

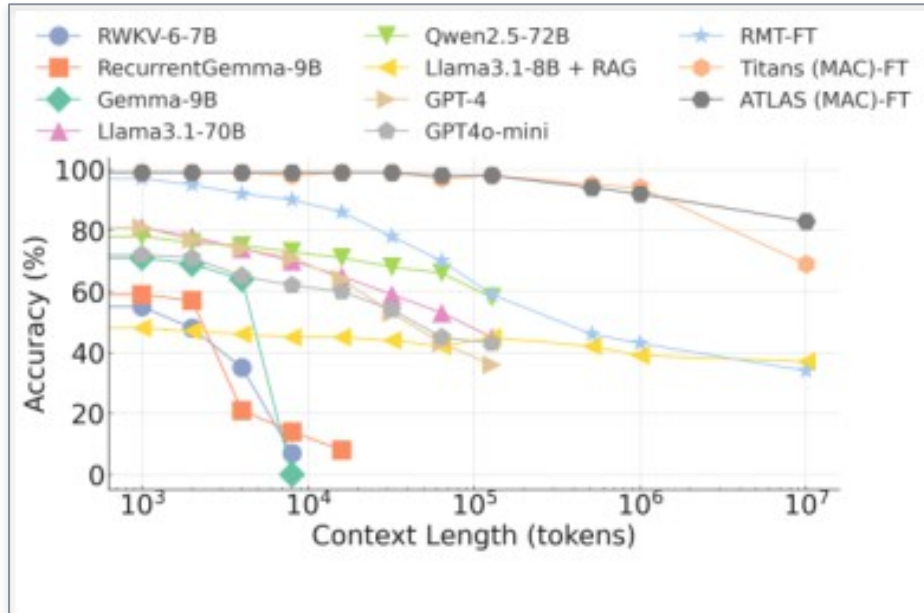
용량 확장 + Muon

$\phi_p(k)$ 로 차원을 $d \rightarrow d^p$ 로 올려 저장 용량 $\uparrow$. 학습은 Muon(근사 2 차) 로 곡률을 활용 .

한 줄 요약

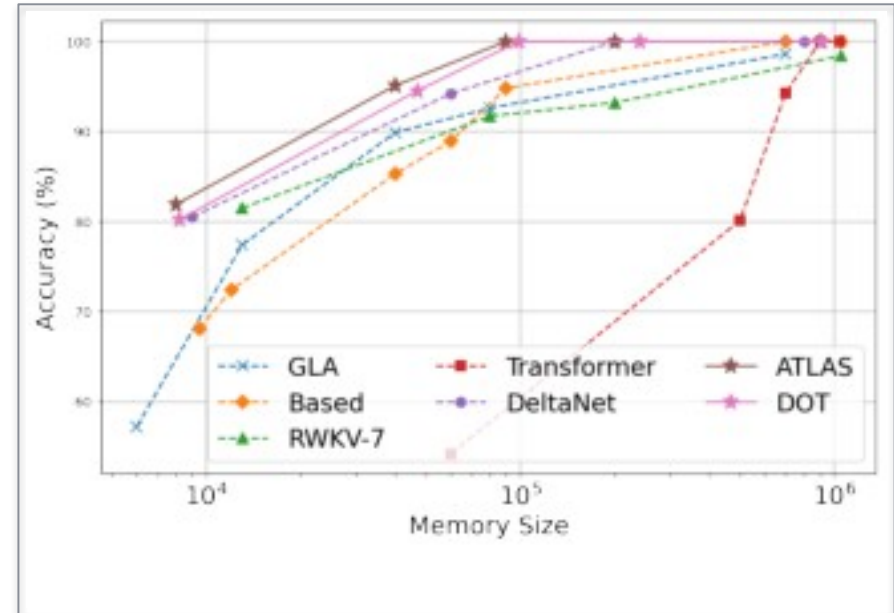
Titans 가 ' 무엇을 기억하나 (연상)' 를 열었다면 , Atlas 는 ' 얼마나 많이 · 문맥 단위로 기억하나 (용량)' 를 키운다 . Miras 의 architecture · algorithm 축을 밀어붙인 결과 .

Titans 를 넘어서고 , 용량이 실제로 커진다



BABILong: Atlas 가 Titans 성능을 초과 , 초장문에서 효과적

출처 : 2505.23735 Fig.4



associative memory recall: feature map 으로 저장 용량이 늘어남을 확인

출처 : 2505.23735 Fig.7

읽는 법

왼쪽 = 같은 벤치에서 Titans 대비 향상 (용량 · Omega 의 효과). 오른쪽 = 저장할 연산 쌍이 많아져도 회상 유지 → '용량 확장' 이 말뿐이 아님을 직접 보여줌.

TNT — 학습 레시피와 serving 구조를 함께 설계

핵심 발명 / 주장

1

train chunk 와 serve chunk 가 다르면 품질이 붕괴한다 (핵심 관찰)

C=64 로 학습한 550M Titans 를 chunk=1 로 서빙하면 성능 급락 — 학습 / 추론 불일치

2

해법 : train chunk(큰) 와 serve chunk(작은) 를 decouple 하고 2-stage 로 정렬

큰 chunk 로 싸게 병렬 학습 → 2 단계로 작은 serve chunk 에 맞춰 재정렬

3

hierarchical memory: global(prefill, 병렬) / local(decode, 세밀)

긴 문맥은 global 로 한 번에 , 생성은 local 로 토큰별 — 병렬성과 정밀도를 모두

4

Q-K projection 추가로 메모리 읽기 / 쓰기 표현력 보강

chunk 정렬과 함께 serving 품질을 끌어올리는 보조 장치

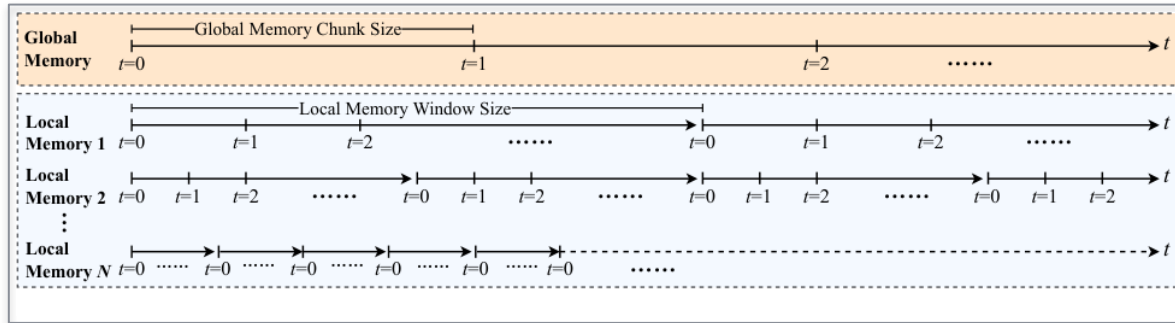
논거 흐름

deep memory 를 큰 chunk 로 싸게 훈련 → 그런데 serve chunk 가 다르면 붕괴 (2.6 배 악화) → 2-stage 로 정렬 → 계층 메모리로 병렬 + 세밀 → serving 구조 정의

증명 (뒤에서 논문 그림 · 표로)

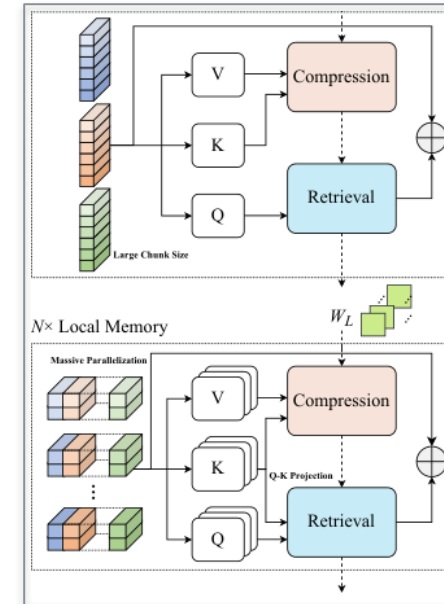
chunk sweep 의 V자 곡선 (Fig.2), 최대 17 배 속도 (Fig.4/5), 품질 표로 train/serve 정렬 효과 입증

메모리 계층 (global/local) + 2-stage 정렬



메모리 계층 다이어그램 : 같은 t 의 갱신을 계층적으로 (global↔local)

출처 : 2511.07343 Fig.1



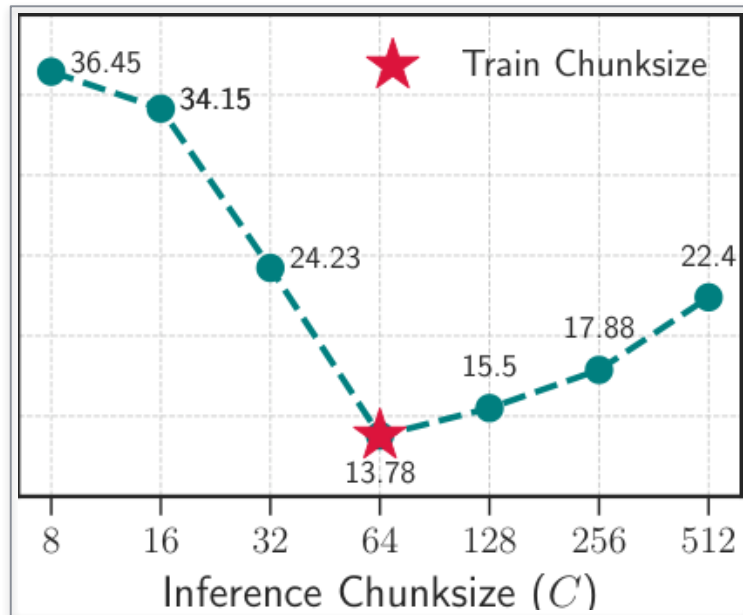
Stage 1 아키텍처 개요 — 큰 chunk 병렬 학습

출처 : 2511.07343 Fig.3

왜 2-stage인가

Stage1: 큰 chunk 로 저비용 병렬 학습 (throughput 확보). Stage2: serve chunk(작게) 에 맞춰 재정렬해 train/serve 불일치를 제거 . 이렇게 해야 ' 싸게 학습 + 정확한 서빙 ' 이 동시에 된다 .

chunk 정렬이 품질을, 계층이 속도를 만든다



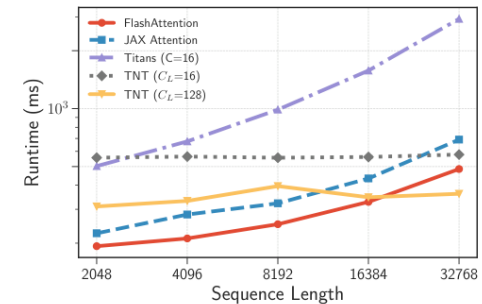
serve chunk 민감도 (V 자): train chunk 와 맞을 때 최적, 어긋나면 급락

출처: 2511.07343 Fig.2

TNT	JAX	{8}	2.54	7.68×
TNT	JAX	{16}	1.65	11.78×
TNT	JAX	{32}	1.22	15.92×
TNT	JAX	{64}	1.12	17.37×
TNT	JAX	{128}	1.16	16.75×

As sequence length increases, TNT's superior scalability creates a crossover point where it becomes faster. This efficiency is most pronounced at very long contexts; for instance, at a sequence length of 32K, a implementation of TNT ($C_L = 128$) outperforms even the highly optimized FlashAttention kernel, confirming y for demanding long-context training scenarios.

Quality Comparison. We next translate -step runtime gains into a practical time-to-arg. As shown in Table 1, our TNT framework accelerates the total training time required to reach target model quality. Our best configuration is up to **17.4× faster** than the original Transformer. This efficiency gain is fundamental to the ; for instance, using an identical local memory of 8, TNT is already **7.7× faster** than its counterpart. While competitive with standard transformers in JAX, our implementation does not perform highly optimized baselines like the Transformer with FlashAttention (Dao, Fu, et al. 2022) as an expected result, as TNT currently lacks a self-attention, which we leave for future work. Nonetheless, these results establish TNT as an efficient foundation.



runtime 비교: 시퀀스가 길어질수록 최대 17 배 빠른 서빙

출처: 2511.07343 Fig.4

읽는 법

왼쪽 V 자 = 'train/serve chunk mismatch 가 진짜 품질을 죽인다' 의 직접 증거. 오른쪽 = 계층 메모리로 긴 문맥 서빙이 실제로 빨라짐. 이 논문은 '어떻게 배포하나' 를 처음으로 정면으로 다룬다.

HOPE — 모든 부품이 '어떤 주파수의 메모리' 인 아키텍처

핵심 발명 / 주장

1

Expressive Optimizers: optimizer 는 gradient 를 압축하는 associative memory 다
momentum·Adam·Muon = 메모리의 특수형 → 더 표현력 큰 optimizer(DGD·M3) 로 승격

2

Self-Modifying Titans: 투영 · 게이트까지 메모리로 만들어 자기 갱신 규칙을 스스로 수정
6 개 메모리 {k,v,q,η,α,mem} 중 q 만 static, 나머지는 test-time 에 갱신

3

CMS(Continuum Memory System): long/short 를 '연속 주파수' 스펙트럼으로 일반화
여러 주파수의 MLP 블록을 이어 붙임 — TNT 의 global/local 을 연속체로 확장 . 합 = HOPE

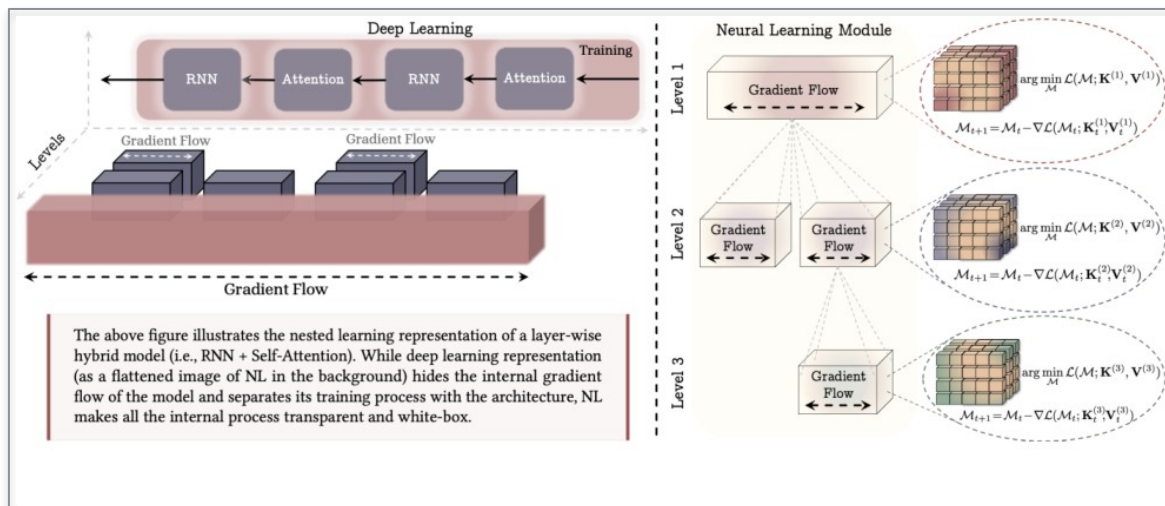
논거 흐름

backprop 도 self-referential memory → optimizer 도 memory → 그림 architecture 와 optimizer 는 같은 것의 다른 레벨 → 레벨을 더 쌓자 (higher-order ICL) → self-mod + CMS = HOPE. ICL 은 창발이 아니라 ≥ 2 레벨의 structural 결과

증명 (뒤에서 논문 그림 · 표로)

760M/1.3B 벤치 (Transformer++·RWKV7·DeltaNet·Titans 대비), BABILong, 메모리 레벨 ablation, M3 optimizer(ViT)

학습 자체를 '중첩된 최적화 (= 메모리) 의 층' 으로 본다



Nested Learning: 모델과 그 학습절차를 중첩된 최적화 문제들의 집합으로

출처 : 2512.24695 Fig.2

핵심 재해석

optimizer 가 하는 일 (과거 gradient 를 모아 다음 스텝을 정함) = '연상 메모리' 다 . 그럼 momentum 은 선형 메모리 , Adam · Muon 도 메모리의 특수형 .

optimizer $\equiv$ associative memory over gradients

그래서 무엇이 열리나

optimizer 를 '더 표현력 있는 메모리' (Deep GD · delta-momentum · M3) 로 바꿀 수 있고 , 이 최적화 층을 여러 개 쌓으면 higher-order in-context learning 이 된다 .

ICL 에 대한 주장

in-context learning 은 큰 모델에서 '창발' 하는 마법이 아니라 , 최소 2 개 레벨의 중첩 최적화를 가지면 구조적으로 (structural) 따라 나오는 성질이다 — Fig.7 의 레벨 ablation 이 이를 뒷받침 .

투영 · 게이트까지 메모리 — 자기 갱신 규칙을 스스로 수정



↑ 6 개 메모리 중 q 만 static(고정), 나머지 5 개는 추론 중 갱신됨

무엇이 새로운가

Titans 는 본체 메모리 M_mem 만 갱신했다 . HOPE 는 ' 무엇을 key/value 로 삼을지 (투영) ' 와 ' 얼마나 기억 · 잊을지 (게이트) ' 까지 메모리로 만들어 test-time 에 함께 갱신한다 .

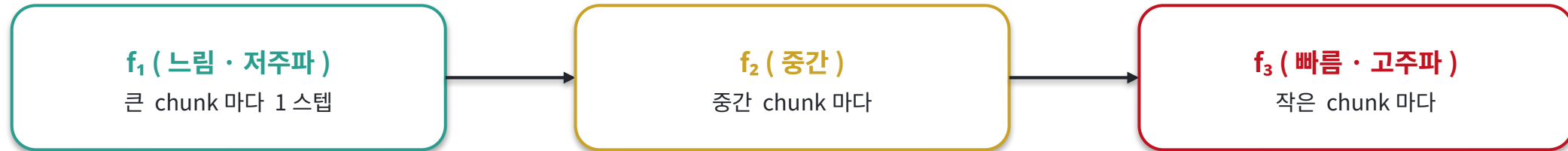
어떻게 갱신하나

target 을 스스로 만든다 (self-target: $\hat{v}_\square = M_\square(v_t)$). 갱신은 DGD-with-weight-decay, gradient 는 full backprop. 출력은 갱신 전 (chunk 시작) 메모리로 읽는다 .

의미

Titans 의 ' 고정된 투영 ' 을 풀어 , 모델이 자신의 학습 규칙 (무엇을 어떻게 기억할지) 을 데이터에 맞춰 바꾸게 만든 것 . q 를 고정으로 남긴 건 안정적 읽기 기준을 유지하기 위함 (추론).

long/short 를 ' 연속 주파수 ' 의 메모리 블록들로



forward: $y = MLP^{f_3}(\dots MLP^{f_1}(x))$ · 각 블록은 자기 주파수 경계에서 gradient 를 모아 1 스텝 갱신

TNT와의 관계

TNT의 global/local(2 단계)을 임의 개수의 주파수로 일반화. 느린 블록 = 장기 문맥, 빠른 블록 = 국소 세부. 블록 간 전달은 '가중치 복사'가 아니라 forward 활성화로.

meta-learned 초기상태

각 블록의 '리셋 지점'(초기 상태)은 사전학습으로 meta-learn. 느린 블록이 빠른 블록의 리셋 초기값을 제공. 이건 런타임 diff 전달이 아니라 시작점 설정 (→ B6 Sleep 가 그 diff 전달을 맡음).

HOPE = self-modifying Titans + CMS

자기수정 (어떻게 기억할지) + 연속 주파수 (언제 기억할지)를 합치면, '모든 부품이 어떤 주파수의 메모리'인 하나의 중첩 시스템 = HOPE가 된다.

HOPE 의 렌즈로 앞 4 편을 한 문장씩 다시 읽기

Titans

원래 : 메모리 = gradient 로 학습되는 신경망 (surprise+momentum+forget).

HOPE: HOPE: 그 optimizer(momentum 등) 마저 메모리 → 아키텍처와 optimizer 가 같은 것의 두 레벨.

Miras

원래 : 4 축 (attentional bias · retention · architecture · algorithm) 으로 설계공간을 정의.

HOPE: HOPE: 'algorithm' 축 (momentum/Adam/Muon) 이 사실 메모리 → architecture · algorithm 축을 하나로 통합.

Atlas

원래 : feature map 으로 용량 확장 + Muon(2 차) optimizer 로 메모리 학습.

HOPE: HOPE: Muon 류 = gradient 의 '표현력 큰 메모리' → M3(더 표현력 있는 optimizer) 로 일반화.

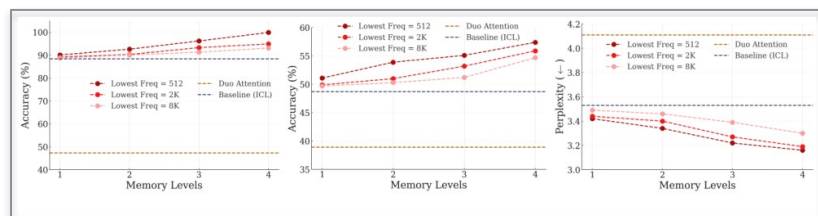
TNT

원래 : global/local(장기 / 단기) 2 단계 메모리 + train/serve chunk 정렬.

HOPE: HOPE: long/short 를 CMS 의 '연속 주파수' 스펙트럼으로 확장 — 2 단계가 아니라 연속체.

한 문장 : HOPE 는 '모든 부품 (투영 · 게이트 · optimizer · 장단기 메모리) 이 어떤 주파수의 메모리' 라는 하나의 중첩 시스템으로 앞 4 편을 통합한다.

레벨이 ICL 을 , 표현력 큰 optimizer 가 학습을 개선한다



메모리 레벨 ↑ → ICL(NIAH) 성능 ↑ (structural)

출처 : 2512.24695 Fig.7

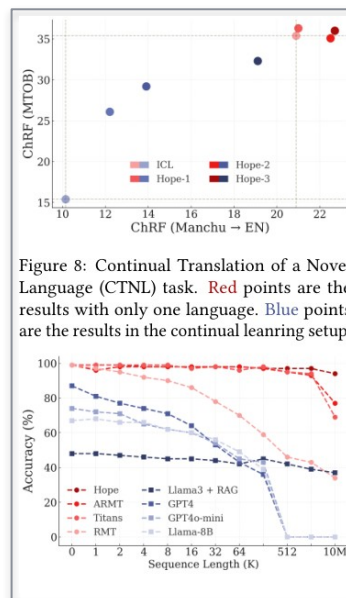
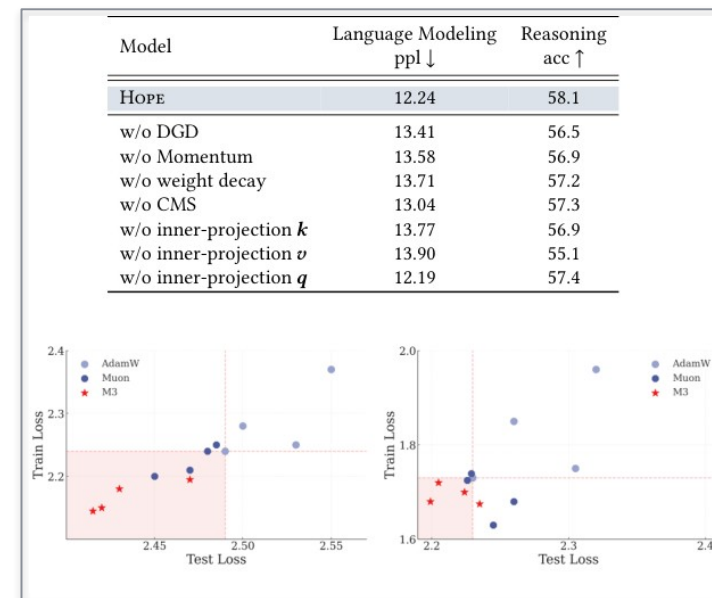


Figure 8: Continual Translation of a Novel Language (CTNL) task. Red points are the results with only one language. Blue points are the results in the continual learning setup.

BABILong: HOPE 가 대형 모델과 견줄

출처 : 2512.24695 Fig.9



M3 optimizer: ViT 에서 AdamW·Muon 대비 개선

출처 : 2512.24695 Fig.11

읽는 법

왼쪽 = 중첩 레벨을 늘릴수록 in-context 성능이 오른다 (ICL=structural 주장의 증거). 가운데 = 초장문 추론에서 경쟁력. 오른쪽 =optimizer 를 메모리로 보는 관점이 실제 학습 (ViT) 에서도 이득.

Sleep — 자는 동안 지식을 ' 위로 ' 옮겨 파라미터를 키운다

핵심 발명 / 주장

1

wake/sleep lifecycle: 추론 (wake) 과 오프라인 통합 (sleep) 을 분리

sleep 은 별도 오프라인 단계 — 이때는 서빙하지 않고 그동안 배운 것을 정리 · 통합한다

2

Memory Consolidation = Knowledge Seeding: 배운 지식을 상위로 distill 해 low-rank expert 를 ' 성장 '

NL 의 online consolidation 은 초기상태 (reset) 만 meta-learn — ' 학습된 diff 의 위쪽 전달 ' 은 못 했다 . Sleep 가 그것을 함

3

Dreaming: 스스로 만든 데이터로 자기개선 (SEAL 식 RL)

teacher 가 ' 꿈 '(자기 생성 시퀀스) 을 만들고 RL 로 강화 — 외부 데이터 없이 능력 향상

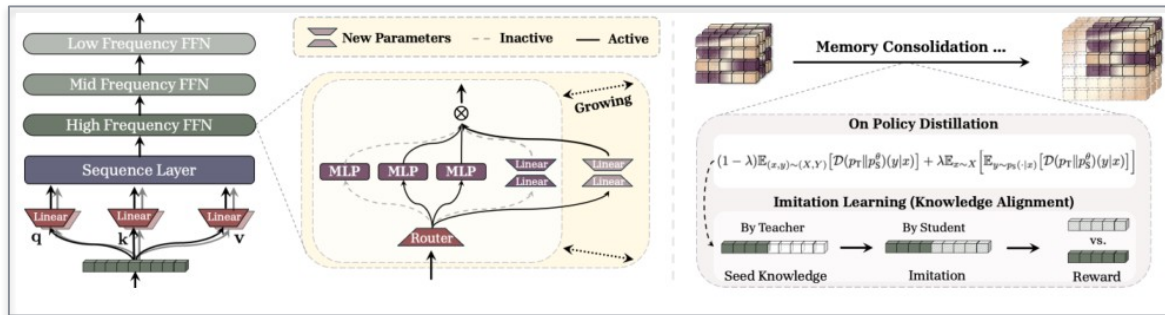
논거 흐름

NL online consolidation(초기상태 meta-learn + circle-back) 은 고정 용량 한계 → 오프라인으로 파라미터 성장 → GKD(on-policy)+LTI(RL) → 새 low-rank expert 만 학습 (나머지 freeze) → synaptic pruning → Dreaming 자기개선

증명 (뒤에서 논문 그림 · 표로)

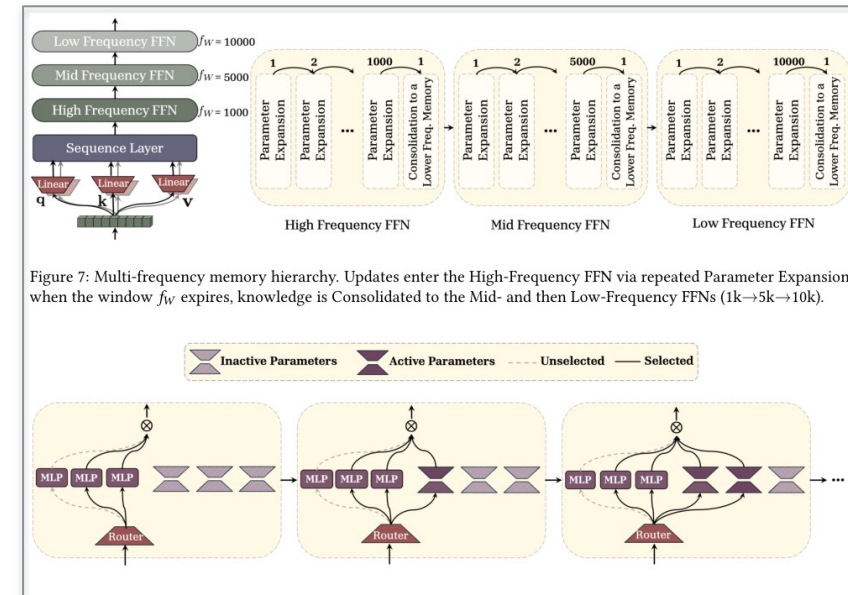
class-incremental(CLINC/Banking/DBpedia) 지속학습 , 메모리 레벨 효과 , BABILong 등으로 통합 효과 입증

느린 블록에 새 저차원 expert 를 키워 지식을 위로 옮긴다



Memory Consolidation: 모델이 스스로 파라미터 수를 늘려 용량을 확장

출처 : 2606.03979 Fig.2



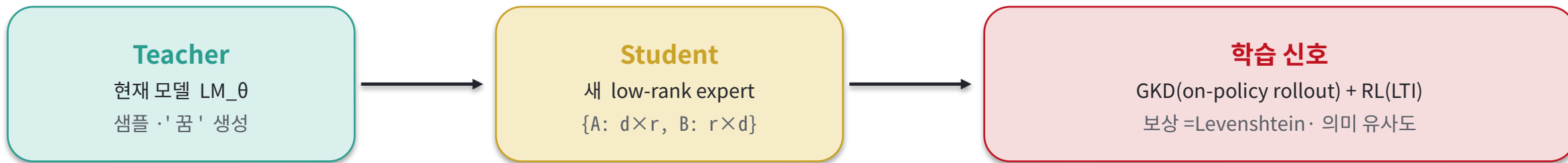
Sleep cycle마다 router 가 새 expert 를 선택 · 갱신 (원→오른쪽)

출처 : 2606.03979 Fig.8

B5(HOPE) 와의 연결 — 빠진 조각

HOPE 의 CMS 는 ' 느린 블록이 빠른 블록의 리셋 초기값 ' 을 줄 뿐 (런타임 diff 전달 아님). Sleep 는 오프라인에서 빠른 (고주파) 메모리가 배운 것을 새 low-rank expert 로 만들어 느린 블록에 ' 실제로 ' 었는다 = 위쪽 전달 .

전부 오프라인 · 자기생성 : teacher→student, GKD + RL



Knowledge Seeding (통합)

teacher 가 만든 on-policy 데이터로 student 를 GKD. 새 expert 만 backward(나머지 freeze). router 가 새 expert 를 활성화 .

LTI — RL 파트

정답 시퀀스와의 Levenshtein/ 의미 보상으로 policy-gradient. 미분 불가능한 목표를 RL 로 .

Dreaming (자기개선)

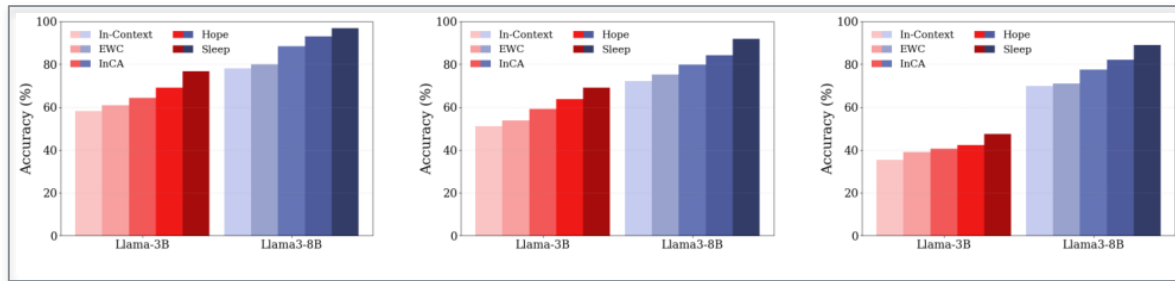
teacher 가 자기 생성한 ' 꿈 ' 으로 SEAL 식 편집 · 강화 — 외부 라벨 없이 능력을 끌어올림 .

reset = (de)activation

새 주기 시작 시 빠른 블록의 이전 expert 들을 비활성화 (pruning) 해 간섭을 막는다 .

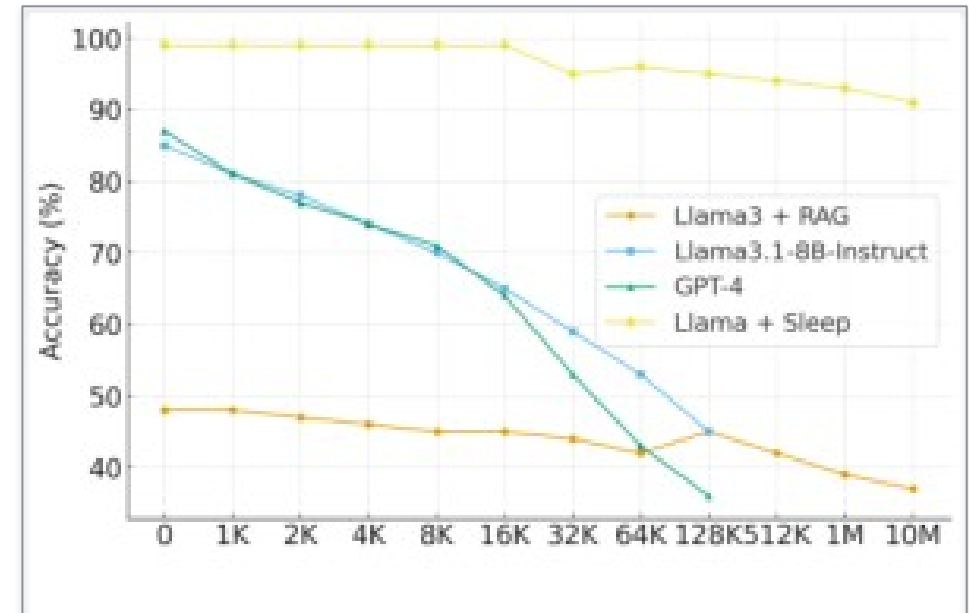
핵심 : 학습 데이터가 전부 모델 자신에게서 나온다 (self-generated) — 외부 코퍼스 추가 없이 지속 학습 .

지속 학습에서 망각을 줄이고 성능을 유지한다



class-incremental(CLINC 등): sleep 통합이 이전 클래스 망각을 완화

출처: 2606.03979 Fig.3



BABILong: 오프라인 통합 후에도 장문 추론 능력 유지 · 향상

출처: 2606.03979 Fig.6

읽는 법 · 6 편의 도착점

Titans 가 연 'test-time 학습' 은 → Sleep 에서 'wake= 빠른 학습 / sleep= 느린 통합' 의 완결된 생애주기가 된다 . 고정 용량의 한계를 파라미터 성장으로 넘은 것이 이 논문의 마지막 퍼즐 .

PART C

**System modeling (우리 기여) — 이 배포는 서빙에서
얼마나 드나**

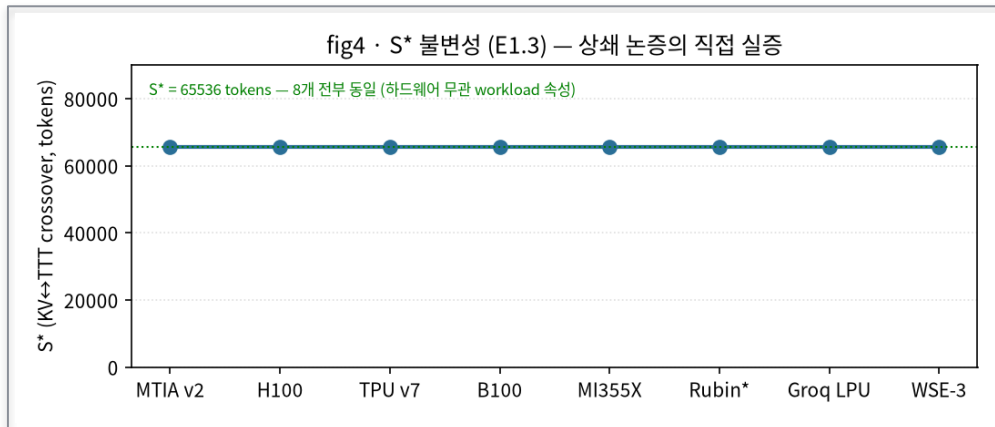
학습은 compute-bound, 디코드는 memory-bound

학습 (training) — compute-bound

chunk C 를 크게 잡아 병렬 → 연산 / 바이트 (arithmetic intensity) 가 높다.
roofline 의 오른쪽 (연산 한계) → 더 빠른 연산기가 이득.

디코드 (decode) — memory-bound

chunk=1, 토큰마다 '전체 상태 (state+weights)' 를 read-modify-write.
arithmetic intensity 가 낮다 → roofline 의 왼쪽 (대역폭 한계).



$S^*(KV↔TTT \text{ 교차점}) = 65536 \text{ tokens}$ — 8 개 가속기 전부 동일 = 하드웨어가 아니라 workload 의
성질

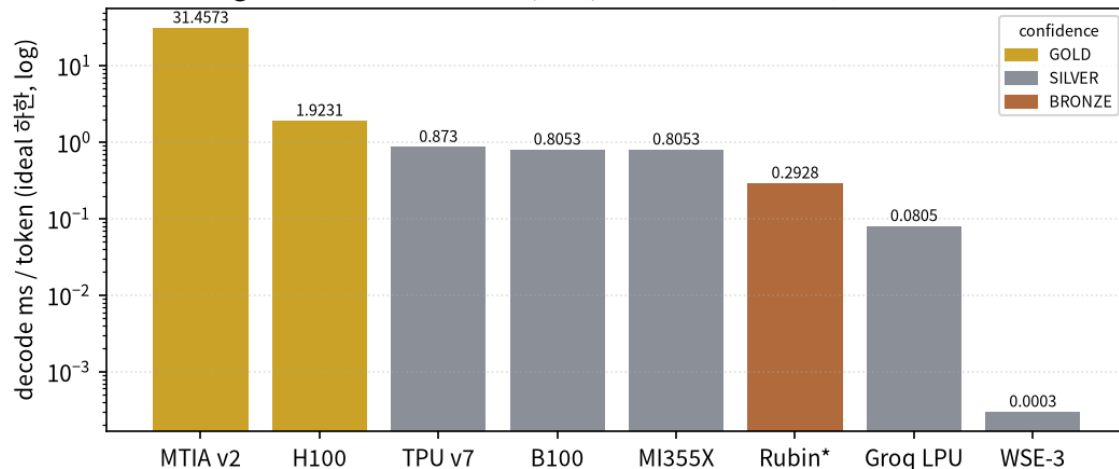
왜 중요한가

메모리 상태를 갱신하는 이 계열은 디코드에서 구조적으로 memory-bound 다. 그리고 그 경계 (S^*) 는 하드웨어를 바꿔도 변하지 않는다 (invariant). 즉 '어느 칩을 써도' 디코드 비용은 메모리 대역폭이 지배한다.

→ 이후 슬라이드 : 실제 8 칩 측정 · HOPE block roofline · 메모리 기술 (zHBM/PIM/SRAM) · 청사진.

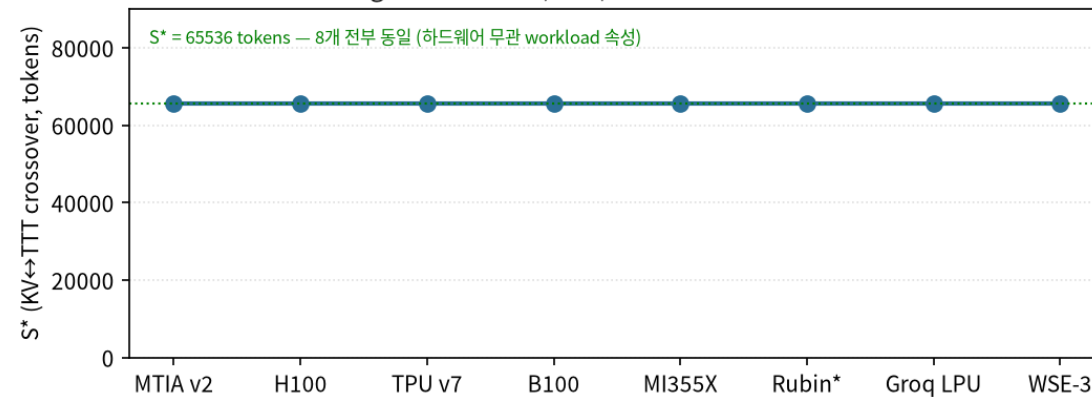
5 자릿수 스프레드 , 그러나 대부분 memory-bound

fig1 · 토큰당 decode 비용 (E1.1) — 5자릿수 스프레드, state 배치가 축



토큰당 decode 시간 (ideal 하한, log) — MTIA v2 ~31ms 에서 WSE-3 ~0.0003ms 까지

fig4 · S* 불변성 (E1.3) — 상채 논증의 직접 실증

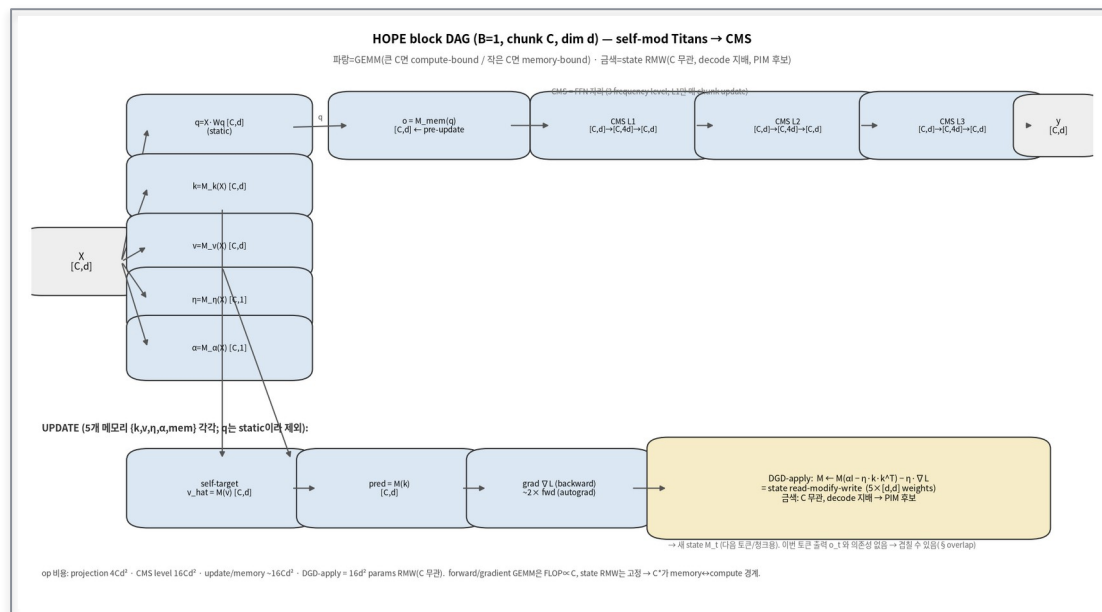


S* 불변 : 8 개 아키텍처가 같은 교차점 — workload 고유 성질

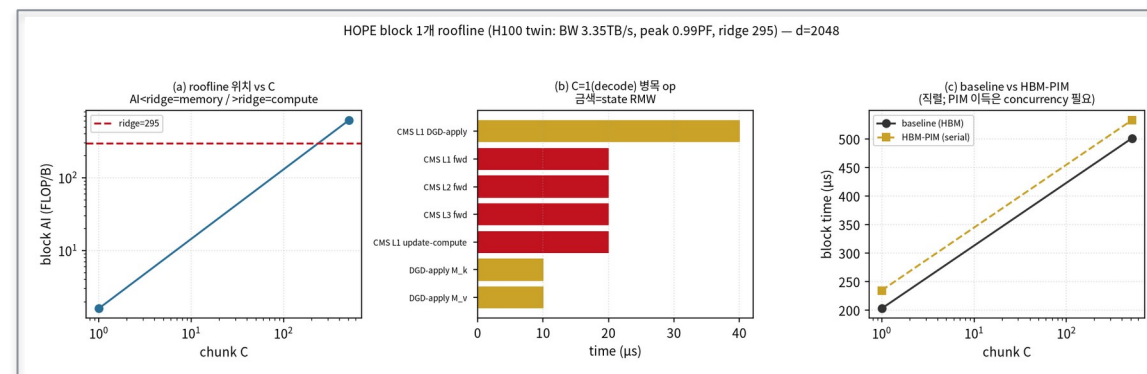
읽는 법

8 칩 중 7 개가 memory-bound(대역폭이 결정). 예외는 WSE-3(Cerebras) — 거대한 on-chip SRAM 으로 상태를 상주시켜 compute 쪽으로 넘어가는 'knee' 가 생긴다 . 각 칩 spec 은 HAT hw twin 으로 검증 (die/PE 구조까지).

1 개 HOPE block 을 실제 수치로 — 어디가 병목인가



1 HOPE block(self-mod Titans→CMS) 의 전 연산 DAG (B=1, chunk C, d=2048)

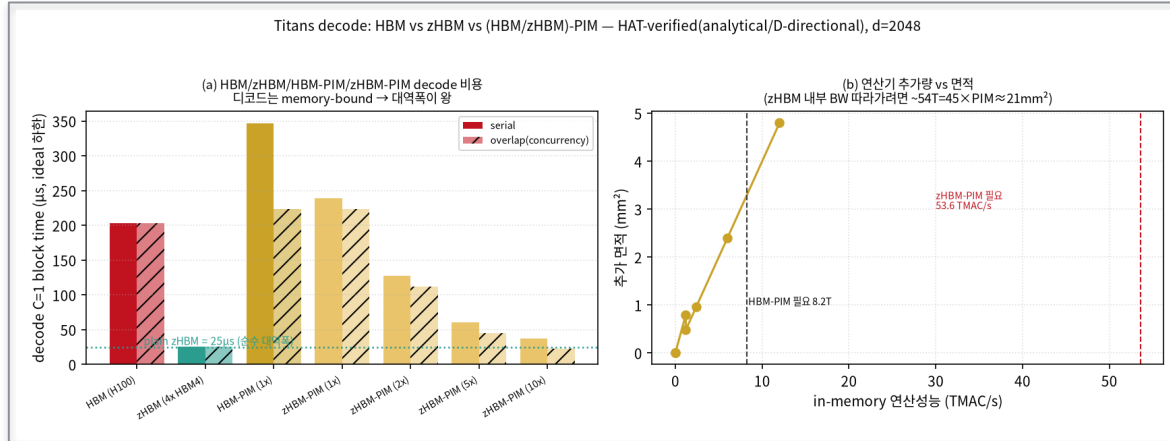


roofline · op 별 시간 · PIM 비교 (BW 3.35TB/s, peak 0.99PF, ridge 295)

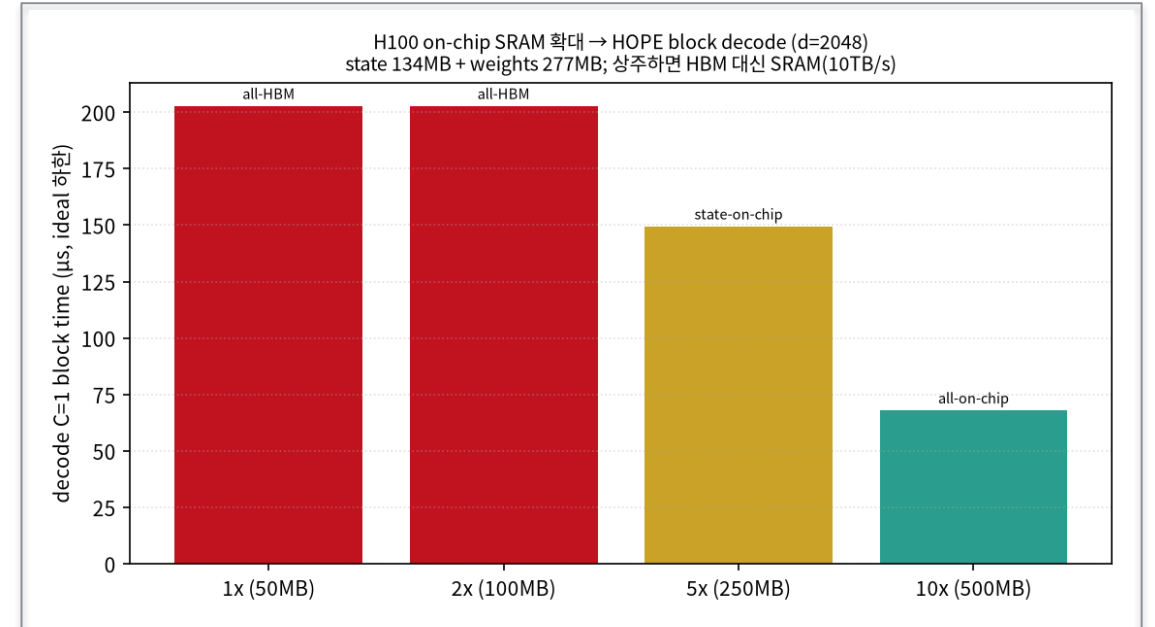
병목

C=1 디코드는 roofline 왼쪽 (memory-bound). op 별로 보면 CMS/DGD-apply 의 state read-modify-write 가 지배적 . state($\approx 134\text{MB}$) + weights 를 매 토큰 HBM(3.35TB/s) 에서 읽어야 해 block 당 $\sim 203\mu\text{s}$ (ideal 하한).

memory-bound 라면 답은 '대역폭' 과 '상주'



HBM/zHBM/HBM-PIM/zHBM-PIM decode 비용 + zHBM 내부 연산 추가 면적

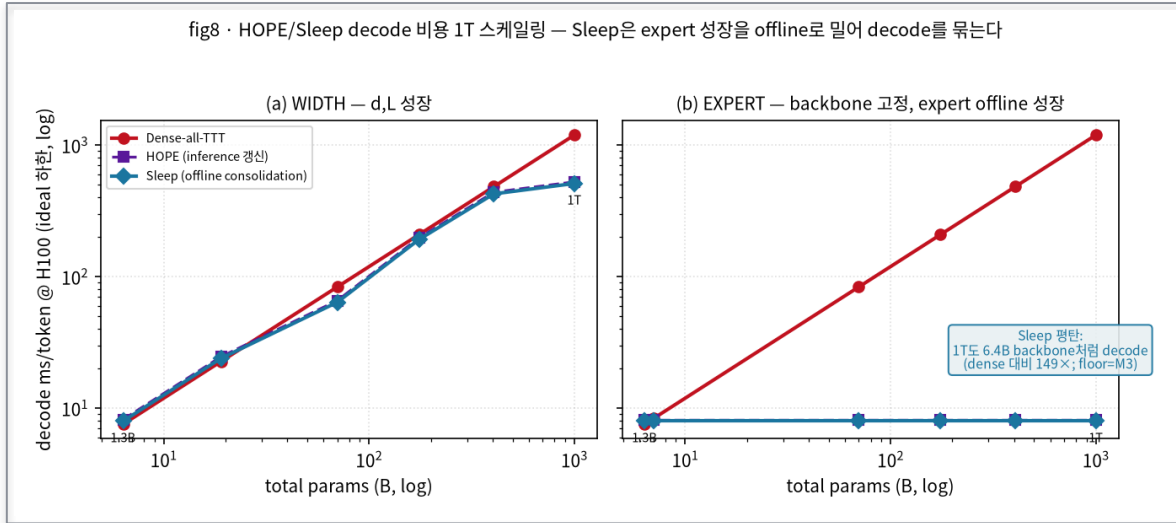


on-chip SRAM 1x→10x: state 가 상주하는 순간 (5x~) HBM 대신 SRAM

결론

zHBM(대역폭 ↑) 이 HBM-PIM(in-bank 연산이 너무 느림) 보다 유리 . SRAM 은 ' 문턱 ' 구조 — state($\approx 134\text{MB}$) 가 on-chip 에 들어가는 5x(250MB) 부터 이득 (203→150 μs), 10x(500MB) 면 weights 까지 전부 상주 .

논문의 방향 × 모델링 결과 = 어디로 가야 하나



HOPE(메모리 확장)+Sleep(파라미터 성장) 스케일링 — 상태 · 파라미터가 함께 자란다

한 줄

이 계열을 키울수록 서빙은 ' 메모리 문제 ' 가 된다 — 대역폭 · 상주 · 소프트웨어가 스케일의 열쇠 .

① 논문 방향이 memory 압력을 키운다

HOPE(self-mod+CMS) 로 state ↑ , Sleep(expert 성장) 로 파라미터 ↑ → 디코드 memory-bound 가 구조적으로 심해진다 .

② 대역폭 우선

zHBM 류 고대역폭 메모리가 1 순위 지렛대 (PIM 보다 유리) .

③ state 상주 설계

on-chip 용량을 state 가 들어갈 문턱 이상으로 — Cerebras 의 knee 가 그 증거 .

④ 소프트웨어 효율이 프론티어

cross-vendor 소프트웨어 효율 (HATIR) 이 남은 최대 변수 .

6 편을 한 흐름으로 , 그리고 서빙 비용까지

Titans	test-time 에 학습되는 신경망 메모리 (surprise+momentum+forget) 를 연다 .
Miras	그 설계를 4 축 공간으로 조직화 — 무엇을 · 무엇을 잊고 · 어떤 구조로 · 어떤 알고리즘으로 .
Atlas	문맥 단위 기억 (Omega) 과 용량 확장 (feature map)+Muon 으로 표현력을 키운다 .
TNT	train/serve chunk 정렬과 global/local 계층으로 ' 어떻게 배포하나 ' 를 답한다 .
HOPE	optimizer=memory·self-mod·CMS 로 앞 4 편을 하나의 중첩 시스템으로 통합 .
Sleep	wake/sleep 생애주기로 지식을 오프라인에서 위로 옮겨 파라미터를 키운다 .
우리 기여	이 배포는 디코드에서 memory-bound — 8 칩 측정 +HOPE roofline+zHBM/SRAM 로 비용과 청사진을 냈다 .